DRIFTGUARD: Machine Learning Based Drift and Security Anomaly Detection in Software System

A Report submitted in partial fulfillment of the requirements for the Degree of

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

by

Ashish Datta Reddy Bavanam	**(2211CS010034)**
Surya Prakash Anapala	**(2211CS010023)**
Kavya Batthula	**(2211CS010065)**
Venkata Prashanth Badveeti	**(2211CS010046)**

Under the guidance of

Dr. xxxxxxxxxx

Associate Professor

Department of Computer Science and Engineering

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2026

DRIFTGUARD : Machine Learning Based Drift and SecurityAnomaly Detection in Software System

A Report submitted in partial fulfillment of the requirements for the Degree of

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

by

Ashish Datta Reddy Bavanam	**(2211CS010034)**
Surya Prakash Anapala	**(2211CS010023)**
Kavya Batthula	**(2211CS010065)**
Venkata Prashanth Badveeti	**(2211CS010046)**

Under the guidance of

Dr. xxxxxxxxxx

Associate Professor



Department of Computer Science and Engineering

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2026

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project report entitled **"DRIFTGUARD: Machine Learning Based Drift and Security Anomaly Detection in Software System"** submitted by Ashish Datta Reddy Bavanam (2211CS010034), Surya Prakash Anapala (2211CS010023), Kavya Batthula (2211CS01 0065), Venkata Prashanth Badveeti (2211CS010046) towards the partial fulfillment for the award of Bachelor's Degree in Computer Science and Engineering. Malla Redd yUniversity, Hyderabad, is a record of Bonafide work done by him/ her. The results embodied in the work are not submitted to any other University or Institute for award of any degree or diploma.

INTERNAL GUIDE

xxxxxxxxxxxxxxxxxx

DEAN-CSE

Dr. A.V.L.N. Sujith

External Examiner

MALLA REDDY UNIVERSITY
(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science and Engineering

DECLARATION

We hereby declare that the project report entitled **"DRIFTGUARD: Machine Learning Based Drift and Security Anomaly Detection in Software System"** has been carried out by us and this work has been submitted to the Department of Computer Science and Engineering, Malla Reddy University, Hyderabad in partial fulfillment of the requirements for the award of degree of Bachelor of Technology. We further declare that this project work has not been submitted in fullor part for the award of any other degree in any other educational institutions.

Place: Hyderabad

Date:

Ashish Datta Reddy Bavanam **2211CS010034**

Surya Prakash Anapala **2211CS010023**

Kavya Batthula **2211CS010065**

Venkata Prashanth Badveeti **2211CS010046**

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the successful completion of this project report. Firstly, we would like to express our heartfelt thanks to **Mr. Chamakura Malla Reddy, Chairman**, and **Dr V. S. K. Reddy, Vice-Chancello**r, for their visionary leadership and unwavering commitment to academic excellence.

We would also like to express our deepest appreciation to our project guide, Dr.**xxxxxxxxx, Associate Professor, Department of Computer Science and Engineering**, whose invaluable guidance, insightful feedback, and constant support have been instrumental throughout the course of this project and in achieving successful outcomes.

We sincerely thank **Dr. A.V.L.N. Sujith, Dean, CSE, School of Engineering**, for his encouragement and support throughout our academic journey.

We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

Ashish Datta Reddy Bavanam 2211CS010034
Surya Prakash Anapala 2211CS010023
Kavya Batthula 2211CS010065
Venkata Prashanth Badveeti 2211CS010046

ABSTRACT

Software systems that exist today are in a dynamic environment. As CI/CD practices are being widespread, the frequency of updates is high with changes in system behaviour being continuous. The given changes may cause performance drift or even reveal hidden security threats that conventional rule-based monitoring frameworks frequently cannot identify in real time.This paper introduces an adaptive machine learning system that aims at tackling the presented issues identifying both the behavioural drift and security-related anomalies. The proposed design integrates several features, such as a hybrid anomaly detector which uses an Autoencoder and also methods to give a clue on why particular alerts were fired, as well as updating dynamically its knowledge about typical system behaviour. Simulated update scenarios and controlled drift conditions indicate that the approach can detect a broader set of anomalies than traditional threshold-based approaches do, and can also produce fewer alerts when the system is performing normally on the whole, the system can detect more kinds of anomalies than traditional methods which use threshold-based approaches, and reduce unwanted alerts due to natural system fluctuations. This piece of work leads to the creation of the monitoring solutions capable of evolving together with the systems that they are meant to monitor.

TABLE OF CONTENTS

DESCRIPTION	PAGE NUMBER
ABSTRACT	VI
Chapter 1 Introduction	
1.1 Outline of Project	1
1.2 Problem Definition	2
Chapter 2 Literature Survey	
2.1 Survey	3
2.2 Previous Studies and Existing System	4
Chapter 3 Project Objectives	
3.1 Objective of the Project	7
3.2 Limitations of the Project	8
Chapter 4 Design and Methodology	
4.1 Proposed System	11
4.2 Architecture	15
4.3 Modules	19
4.4 Methods and Algorithms	21
4.5 Structural Design	23
4.6 System Design	25
4.7 Data Flow in the System	27
4.8 UML Diagram	29
Chapter 5 Testing	
Need of Testing	34
Chapter 6 Code	35
6.1 System Implementation and outputs	43
Chapter 7 Results and Discussion	
7.1 Results	48
7.2 Discussion	49
7.3 Existing system vs Proposed System	50

Chapter 8 Conclusion

8.1 Project Conclusion **51**

Future Scope **52**

References **53**

LIST OF FIGURES

FIG NO	FIG NAME	PAGE NO
Fig 4.1	Proposed System	13
Fig 4.2	Architecture Diagram	15
Fig 4.3	Data Flow Diagram	27
Fig 4.4	Usecase Diagram	29
Fig 4.5	Class Diagram	30
Fig 4.6	Sequence Diagram	31
Fig 4.7	Activity Diagram	32
Fig 4.8	Deployment Diagram	33
Fig 6.1.1	Home Page	43
Fig 6.1.2	Autoencoder Training	44
Fig 6.1.3	Drift Detection	45
Fig 6.1.4	Performance Comparison	46
Fig 6.1.5	Comparison Graph	47

CHAPTER-1

INTRODUCTION

1.1 Outline of the Project

The present-day condition of the software systems is continually changing. The emergence of CI/CDpractices implies that teams are making updates nearly constantly - fixing bugs, adding features, andtuning performance nearly as fast as it would have been impossible a decade ago. That velocity isterrific to enhance innovation, however, it can be highly hazardous: an execution may fall under thecarpet, a flaw may creep in, and system behaviour may change in a manner that no one predicted. To complicate matters, contemporary production systems produce vast amounts of telemetry per second CPU loads, memory loads, disk loads, network loads, and log hits and that information must be analyzed in real-time. It has no time to group it up and have a look at it later. Concept drift – the gradual or sudden change in statistical fingerprint of normal behaviour of a system - is one of the most difficult issues in this space. The goalposts can be shifted by a new release, a config tweak, as cale-out event or just a change in user traffic patterns. When the monitoring model fails to keep up ,it begins to misinterpret the situation: things that are good appear suspicious and things that are really bad pass undetected. There is another layer of concern of security. On top of performance drift, live systems are persistently at risk - suspicious login attempts, unaccounted processes quietly consuming resources they are not supposed to access and possible data exfiltration, which will pass through simple detection rules. Majority of the classic monitoring tools are based on established thresholds and manually defined alert conditions. They are sensible at picking up the blatant failures, but they are not designed to pick up the type of slow, insidious behavioural change that can lead to a severe event. This paper addresses the two issues directly by suggesting a monitoring framework that combines machine learning-based anomaly detection with statistical drift detection. Instead of using the established rules, the system creates and constantly optimises itself around its model of normal, and re-optimises itself when the data indicates that normal has changed. changed.

To address these challenges, this work introduces **DriftGuard**, an AI-driven framework designed to detect performance drift in desktop applications using an unsupervised learning approach. Unlike traditional monitoring systems that rely on static thresholds and manual configurations, DriftGuard learns the normal behavior of a system directly from baseline performance data and identifies deviations using reconstruction error. By leveraging a lightweight autoencoder model and system-level telemetry collected through efficient monitoring tools, the framework provides an automated and adaptive mechanism for post-update validation. This enables early detection of subtle performance anomalies that may otherwise go unnoticed, thereby improving software reliability and reducing the risk of deploying unstable updates in rapidly evolving development environments.

1.2 Problem Definition

Modern software systems undergo frequent updates due to continuous integration and deployment practices. While these updates aim to improve functionality and performance, they often introduce unintended issues such as performance degradation, resource inefficiencies, or hidden bugs. Detecting these problems is challenging because traditional testing methods are limited and may not capture subtle changes in system behavior.Another major challenge lies in the nature of system telemetry data, which is high-volume, dynamic, and constantly evolving. Performance metrics such as CPU usage, memory consumption, and execution time can vary due to multiple factors, making it difficult to distinguish between normal fluctuations and actual performance drift. Existing monitoring tools typically rely on static thresholds or predefined rules, which are not effective in identifying gradual or complex behavioral changes.

Furthermore, many existing solutions depend on labeled datasets, complex infrastructure, or cloud-based environments, making them unsuitable for standalone desktop applications. There is a need for a lightweight, adaptive, and unsupervised system that can automatically learn normal behavior and detect deviations without manual intervention.

CHAPTER-2

LITERATURE SURVEY AND EXISTING SYSTEMS

2.1 Survey

Several research studies have focused on detecting performance regressions introduced after software updates. Systems like AutoPerf use deep learning techniques, particularly autoencoders, to learn normal execution patterns and identify deviations using hardware performance counters. Similarly, DynamicRegress applies advanced models such as Siamese LSTM networks to compare system behavior across different versions. These approaches demonstrate the effectiveness of machine learning in identifying performance anomalies.In addition to regression detection, general anomaly detection techniques have been widely explored in research. Traditional methods rely on statistical analysis and threshold-based monitoring, while modern approaches use machine learning models such as autoencoders, Isolation Forest, and One-Class SVM. Among these, unsupervised models are particularly useful as they can detect anomalies without requiring labeled datasets, making them suitable for real-world applications.Despite these advancements, many existing solutions are designed for large-scale cloud or enterprise environments and depend on complex infrastructure or low-level hardware monitoring. They often require labeled data, extensive configuration, or continuous workload profiling, which increases implementation complexity. As a result, these methods are not easily adaptable to lightweight desktop applications.

Therefore, there is a clear need for a simple, efficient, and unsupervised approach that can operate in standalone environments. The proposed DriftGuard system addresses this gap by combining lightweight system monitoring with an autoencoder-based anomaly detection model. This approach enables effective detection of performance drift without relying on labeled data or complex infrastructure, making it practical for real-world desktop applications.

2.2 Previous Studies

Autoencoder-Based Anomaly Detection

Autoencoders are widely used in anomaly detection to learn normal data patterns and identify deviations through reconstruction error.

AutoPerf: Regression Testing Using Autoencoders

Authors: Mariani et al. (2019)

Description: Uses deep autoencoders and hardware performance counters to detect software performance regressions.

Limitation: Requires specialized hardware counters and is mainly suited for large-scale systems.

Siamese Network-Based Drift Detection

Advanced neural networks compare behavior between system versions.

DynamicRegress: Performance Regression Detection

Authors: Chen et al. (2023)

Description: Uses Siamese LSTM networks to compare performance metrics between software versions.

Limitation: Requires semi-supervised learning and pseudo-label generation, increasing complexity.

Statistical Drift Detection Approaches

Statistical techniques detect distribution changes in data over time.

Learning with Drift Detection (DDM)

Authors: Gama et al. (2004)

Description: Detects drift by monitoring changes in classification error rates.

Limitation: Requires labeled data and is not suitable for unlabeled performance metri**Cresci**

Performance Monitoring with System Metrics

System-level monitoring approaches.

Psutil-Based Monitoring Systems

Authors: Python Community

Description: Collects CPU, memory, and system performance metrics for analysis.

Limitation: Does not provide built-in anomaly detection capability.

Replicator Neural Networks

Early neural-network-based anomaly detection models.

Outlier Detection Using Replicator Neural Networks

Authors: Hawkins et al. (2002)

Description: Uses neural networks to reconstruct input and detect anomalies based on reconstruction error.

Limitation: Limited scalability and performance on large datasets.

Variational Autoencoder-Based Detection

Probabilistic version of autoencoders.

VAE for Anomaly Detection

Authors: Kingma & Welling (2013)

Description: Learns data distribution and detects anomalies probabilistically.

Limitation: Computationally complex and harder to train.

LSTM-Based Time Series Anomaly Detection

Used for sequential data analysis.

LSTM Autoencoder for Anomaly Detection

Authors: Malhotra et al. (2016)

Description: Detects anomalies in time-series data using sequence learning.

Limitation: High computational cost and requires large datasets.

Threshold-Based Monitoring Systems

Traditional monitoring approach.

Static Threshold Monitoring

Authors: Various

Description: Detects anomalies when metrics exceed predefined thresholds.

Limitation: Cannot detect subtle or gradual performance drift.

Hybrid and Integrated Detection Systems.

Gaussian Mixture Models (GMM)

Probabilistic clustering-based approach.

Gaussian Mixture Model for Anomaly Detection

Authors: Reynolds (2009)

Description: Models data as multiple Gaussian distributions to detect anomalies.

Limitation: Assumes data distribution, which may not hold in real-world systems.AI-Based Cyber Threat Intelligence Systems

Transformer-Based Regression Detection

Modern deep learning approach.

DiffBERT for Performance Regression

Authors: Facebook Research (Meta)

Description: Uses transformer models to analyze code changes and predict regressions.

Limitation: Requires large-scale training data and high computational resources.

Rule-Based Intrusion Detection Systems

Traditional cybersecurity approach.

Snort: Network Intrusion Detection System

Authors: Roesch (1999)

Description: Uses rule-based detection to identify known attack signatures.

Limitation: Cannot detect unknown or evolving threats.

CHAPTER-3

PROJECT OBJECTIVES

3.1 Objectives of the Project

1) To develop a lightweight performance monitoring system

To develop a system that collects real-time metrics such as CPU usage memory consumption, and execution time using efficient tools, ensuring minimal overhead on desktop applications.

2) To develop an unsupervised anomaly detection model using autoencoders

To develop a machine learning model that learns normal system behavior from data and identifies deviations without requiring labeled datasets.

3) To develop a mechanism for detecting performance drift after software updates

To develop a method that compares pre-update and post-update system behavior using reconstruction error to identify potential anomalies or regressions.

4) To develop a threshold-based decision system for drift identification

To develop an approach that determines whether drift has occurred by comparing reconstruction error against a dynamically calculated threshold.

5) To develop a user-friendly interface for workflow execution and visualization

To develop an interface that allows users to easily perform data collection, model training, drift detection, and visualize results through charts.

3.2 Limitations of the Project

Limited Feature Set for Performance Monitoring

The current system considers only a small set of performance metrics such as CPU usage, memory consumption, execution time, and their derived ratios. While these are important indicators, they may not capture the complete behavior of complex applications. Other factors such as disk I/O, network activity, thread-level statistics, and application-specific logs are not included, which may limit the system's ability to detect certain types of anomalies..

Static Threshold for Drift Detection

The system uses a threshold derived from baseline reconstruction error to identify anomalies. However, this threshold is static and may not adapt well to dynamic changes in system behavior over time. In real-world scenarios, system performance can fluctuate due to environmental factors, making a fixed threshold less reliable.

Lack of Real-Time Monitoring Capability

DriftGuard currently operates in a batch-processing mode, where data is collected before and after updates and then analyzed. It does not support real-time or continuous monitoring, which limits its ability to detect drift immediately as it occurs in live systems.

No Explainability for Detected Drift

While the system can detect that drift has occurred, it does not provide detailed explanations regarding which specific features contributed most to the anomaly. This lack of interpretability can make it difficult for developers or system administrators to diagnose and fix the underlying issues.

Limited Scalability to Large or Distributed Systems

The current implementation is designed for standalone desktop applications and may not scale efficiently to large-scale or distributed systems such as cloud environments or microservices architectures. Handling high-volume data streams and multiple nodes would require additional infrastructure and optimization.

Sensitivity to Noise and Outliers in Data

The autoencoder may sometimes misinterpret noisy or irregular data as anomalies, leading to false positives. Similarly, if the training data contains unnoticed anomalies, the model may learn incorrect patterns, reducing its detection accuracy.

Focus on Performance Metrics Only

The system is primarily designed to detect performance drift and does not explicitly address other important aspects such as functional bugs, security threats, or logical errors in the application. This limits its scope to performance monitoring rather than comprehensive system validation

Limited Generalization Across Different Applications

The system collects a relatively small number of samples (e.g., 25 observations) for both baseline and post-update analysis. While this simplifies implementation, it may not be sufficient to capture the full variability of system behavior, especially under different workload conditions. A limited dataset can lead to less robust model training and may increase the chances of inaccurate drift detection.

Small Dataset Size and Sampling Constraints

The system collects a relatively small number of samples (e.g., 25 observations) for both baseline and post-update analysis. While this simplifies implementation, it may not be sufficient to capture the full variability of system behavior, especially under different workload conditions. A limited dataset can lead to less robust model training and may increase the chances of inaccurate drift detection.

Absence of Temporal Dependency Modeling

The current implementation treats each data point independently and does not consider the sequential or time-dependent nature of performance data. In real-world systems, performance metrics often follow temporal patterns, where the behavior at one point in time is influenced by previous states. Ignoring these temporal dependencies can limit the system's ability to detect gradual or evolving drift patterns.

For example, a slow increase in memory usage over time may indicate a memory leak, but such a trend might not be detected if each data point is analyzed in isolation. Similarly, periodic spikes in CPU usage may represent normal cyclical behavior rather than anomalies. Without capturing these temporal relationships, the model may either miss important drift patterns or incorrectly classify normal variations as anomalies.To address this limitation, future versions of the system could incorporate time-series models such as LSTM-based autoencoders or recurrent neural networks. These models are capable of learning temporal dependencies and can provide more accurate detection of both sudden and gradual changes in system behavior. Integrating such approaches would significantly enhance the system's ability to handle real-world performance monitoring scenarios.

Lack of Automated Model Retraining and Adaptation

The current DriftGuard system does not include a mechanism for automatic model retraining or adaptation as system behavior evolves over time. Once the autoencoder is trained on baseline data, it remains static and does not update itself based on new patterns or long-term changes in system usage. In real-world environments, software behavior can gradually shift due to factors such as user load variations, configuration changes, or hardware differences. Without periodic retraining, the model may become outdated, leading to decreased accuracy and increased false positives or false negatives. Incorporating an automated retraining pipeline or adaptive learning mechanism would allow the system to continuously update its understanding of "normal" behavior, thereby improving long-term reliability and robustness.

CHAPTER-4
DESIGN AND METHODOLOGY

4.1 Proposed System

The proposed system, **DriftGuard**, is an AI-based performance drift detection framework designed to identify anomalies introduced after software updates in desktop applications. The system addresses the limitations of traditional monitoring approaches by leveraging unsupervised machine learning techniques to automatically learn normal system behavior and detect deviations without requiring labeled datasets or predefined rules. DriftGuard is specifically designed to be lightweight, scalable, and easy to deploy in standalone environments.Unlike conventional monitoring systems that rely on static thresholds, DriftGuard adapts to the system's behavior by learning patterns from baseline performance data. It focuses on detecting subtle performance degradations that may not be captured through manual testing or rule-based alerts. The system operates through a structured pipeline consisting of data collection, model training, drift detection, and result visualization.

System Overview

The DriftGuard system follows a modular architecture with clearly defined components that interact to achieve automated drift detection. The workflow begins with the collection of baseline data from a stable version of the application. This data is used to train an autoencoder model that learns the normal performance characteristics. After a software update, new performance data is collected and compared against the learned patterns using reconstruction error. Based on this comparison, the system determines whether performance drift has occurred.

Performance Monitoring Module

The Performance Monitoring Module is responsible for collecting system-level metrics that represent application behavior. It utilizes lightweight system monitoring tools to capture key performance indicators such as CPU usage, memory consumption, execution time, and derived ratios like CPU-to-memory utilization.

Data Preparation

The collected data is processed and prepared for model training and evaluation. This involves:

1. Loading data using data handling libraries.
2. Converting data into numerical format.
3. Transforming data into tensors for machine learning processing.

Since the system uses an unsupervised approach, no labeling is required. The baseline dataset itself serves as the reference for normal behavior. Proper data formatting ensures compatibility with the autoencoder model and improves training efficiency.

Autoencoder-Based Learning Module

The core of the DriftGuard system is the autoencoder model, implemented using a deep learning framework. The autoencoder consists of two main components:

1. Encoder

The encoder compresses input data into a lower-dimensional representation. It captures essential features of system behavior while reducing redundancy.

2. Decoder

The decoder reconstructs the original input from the compressed representation. The goal is to minimize the difference between input and output.

3. Model Architecture

Input Layer: 4 features (CPU, Memory, Execution Time, Ratio)

Hidden Layer: 2 neurons

Output Layer: 4 neurons

The model uses activation functions such as ReLU and is trained using Mean Squared Error (MSE) loss.

Training Process

The autoencoder is trained using baseline data representing normal system behavior. The training process involves:

- Forward propagation of input data
- Reconstruction of output
- Calculation of reconstruction loss
- Backpropagation to update weights

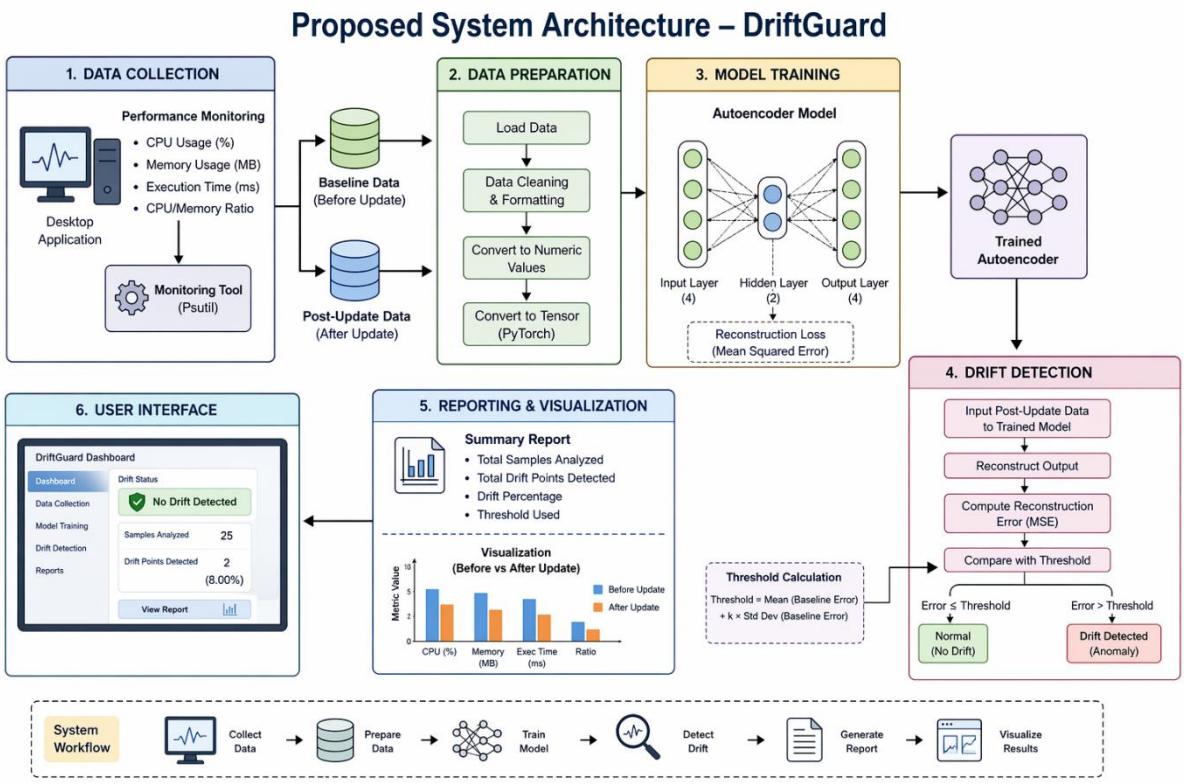


Fig 4.1 Proposed System

Drift Detection Mechanism

After training, the system performs drift detection using post-update data. This includes:

Reconstruction

The trained autoencoder reconstructs each data point from the new dataset.

Threshold Comparison

A threshold is derived from baseline error statistics. If:

Error ≤ Threshold : Normal behavior

Error > Threshold : Drift detected

This mechanism allows the system to identify anomalies without requiring labeled data.

Reporting and Visualization Module

The system generates a detailed drift detection report

System Requirements

The successful implementation and deployment of the **DriftGuard** framework requires appropriate hardware, software, and network infrastructure. Because the system runs real-time Machine Learning inferences and processing, meeting the minimum processing capabilities is essential for smooth operation.

Hardware Requirements

Processor: Intel Core i3 / AMD Ryzen 3

RAM: Minimum 4GB (8GB or higher heavily recommended for training the models).

Storage: 50GB to 100GB of available Disk Space (Solid State Drive preferred for rapid database queries and logging).

Software Requirements

The system is developed using modern open-source technologies, web frameworks, and AI libraries.

Torch Library

Programming language: Python

psutil – for data collection from the system

Autoencoder – for unsupervised learnin

4.2 Architecture

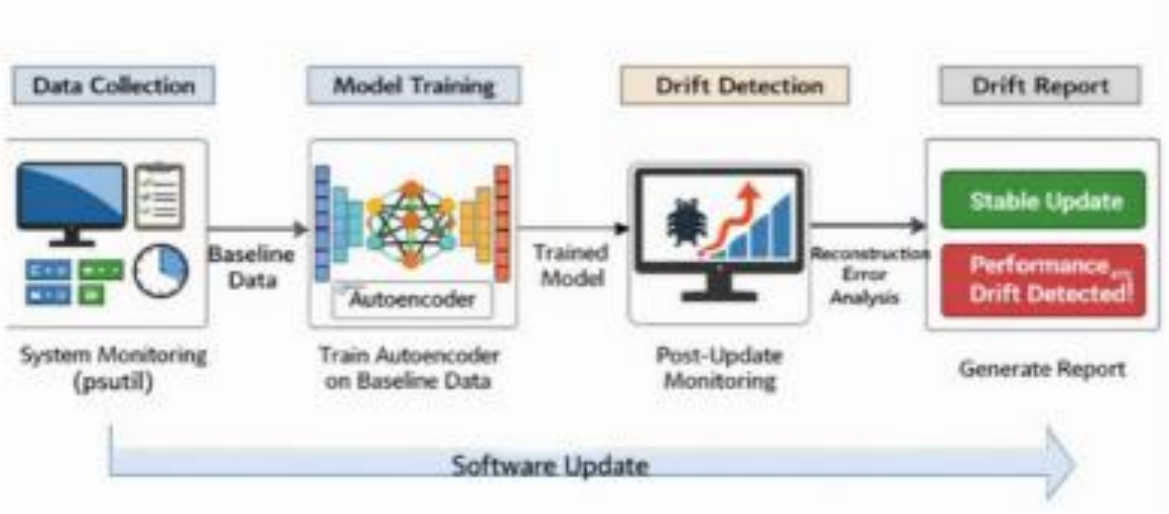


Fig 4.2 Architecture of DriftGuard

The image illustrates the overall workflow of the DriftGuard system, presenting a high-level architecture of how performance drift detection is carried out across different stages of the software lifecycle. The process begins with the Data Collection phase, where system performance metrics such as CPU usage, memory consumption, and execution time are gathered using monitoring tools like psutil, forming the baseline dataset that represents normal system behavior. This data is then passed to the Model Training stage, where an autoencoder is trained on the baseline data to learn the normal performance patterns of the application. Once the model is trained, the system proceeds to the Drift Detection phase after a software update, where new performance data is collected and analyzed. The trained model reconstructs this post-update data, and the reconstruction error is calculated to identify any deviations from the learned baseline behavior. Finally, in the Drift Report stage, the system evaluates whether the detected deviations exceed the predefined threshold and generates a report indicating whether the update is stable or has introduced performance drift. The diagram effectively highlights the end-to-end pipeline, showing how data flows from monitoring to analysis and reporting, and emphasizes the role of AI in automating performance validation after software updates.

The architecture of the DriftGuard system is designed as a modular and layered framework that enables efficient detection of performance drift in desktop applications. It follows a structured pipeline consisting of data collection, model training, drift detection, and reporting. Each module operates independently while maintaining seamless data flow between components. This modular design ensures flexibility, ease of maintenance, and the ability to extend the system with additional features in the future.The first layer of the architecture is the **data collection module**, which is responsible for gathering system-level performance metrics. Using lightweight monitoring tools, the system collects parameters such as CPU usage, memory consumption, execution time, and derived ratios. These metrics are captured both before and after software updates to create baseline and comparison datasets. The collected data is stored locally and serves as the foundation for model training and analysis.

The second layer focuses on the **model training and learning process**, where an autoencoder model is trained using the baseline data. The model learns the normal behavior of the system by minimizing reconstruction error, effectively capturing the underlying patterns in performance metrics. Once trained, the model is saved and reused for analyzing post-update data. This layer forms the core intelligence of the system, enabling it to distinguish between normal and abnormal behavior.

The final layer includes the **drift detection and reporting modules**, where post-update data is passed through the trained model to compute reconstruction error. The system compares this error against a predefined threshold to determine whether drift has occurred. The results are then presented through a reporting and visualization interface, providing summaries, detailed insights, and graphical comparisons of performance metrics. This end-to-end architecture ensures accurate detection, clear interpretation, and user-friendly interaction, making the system practical for real-world deployment.

System Working and Output Components

The working of the DriftGuard system begins with the initialization of the performance monitoring process, where the system prepares to collect baseline data from the desktop application. At this stage, the application is assumed to be in a stable state before any software update is applied. The monitoring module uses lightweight tools to capture system-level metrics, ensuring minimal overhead and accurate representation of normal behavior.The next step involves **baseline data collection**, where key performance parameters such as CPU usage, memory consumption, execution time, and derived ratios are recorded over multiple iterations. This data forms the foundation for understanding how the system behaves under normal conditions. The collected dataset is stored locally in a structured format for further processing. Once the baseline data is collected, the system proceeds to the **model training phase**. Here, the autoencoder model is trained using the baseline dataset. The model learns to compress and reconstruct the input data, capturing the essential patterns of normal system performance. During training, the reconstruction error is minimized to ensure accurate learning.After training is completed, the system saves the trained model and transitions to the **post-update data collection phase**. Following a software update, the same set of performance metrics is collected again under similar conditions. This ensures consistency and allows for meaningful comparison between pre-update and post-update behavior.The next stage is the **data input and reconstruction process**, where the post-update data is fed into the trained autoencoder model. The model attempts to reconstruct the input data based on the patterns it learned during training. Any deviation from normal behavior results in higher reconstruction error values.

Following reconstruction, the system performs **error calculation and analysis**. The reconstruction error for each data point is computed using a suitable metric such as Mean Squared Error (MSE). These error values indicate how closely the new data aligns with the baseline behavior learned by the model.

The system then applies the threshold-based drift detection mechanism. A threshold is calculated from the baseline error distribution, and each post-update data point is compared against this threshold. If the error exceeds the threshold, it is classified as a drift point, indicating abnormal behavior.Once drift detection is completed, the system generates the drift detection output, which includes the total number of samples analyzed, the number of drift points detected, and detailed information about each anomaly. This output helps users understand whether the software update has introduced performance issues.In addition to textual output, the system provides visualization components such as performance comparison charts. These charts display differences in CPU usage, memory consumption, and execution time before and after the update, allowing users to quickly interpret changes in system behavior.Finally, the system presents the results through a user interface dashboard, where users can view drift status, detailed reports, and graphical insights. This output component enhances usability by providing a clear and intuitive representation of the analysis, enabling informed decision-making regarding software stability and performance.

In addition to the core workflow, the system incorporates a model persistence and reuse mechanism, where the trained autoencoder is saved as a file and reused for future drift detection tasks. This eliminates the need to retrain the model every time new data is analyzed, thereby improving efficiency and reducing computation time. The saved model ensures consistency in evaluation and allows the system to quickly process post-update data whenever required. This component plays a crucial role in making the system practical for repeated use in real-world scenarios.

4.3 Modules

The DriftGuard is highly decoupled and divided into several functional modules. Each module performs a specific, optimized task to ensure accurate and extremely efficient threat detection and explanation.

Performance Monitoring Module

This module is responsible for collecting real-time system performance metrics using lightweight tools such as psutil. It gathers key parameters including CPU usage, memory consumption, execution time, and CPU–memory ratio. The data is collected both before and after the software update to ensure consistency. This module acts as the foundation of the system, as the quality of collected data directly impacts the accuracy of drift detection.

Data Handling Module

The data handling module manages the storage and processing of collected performance data. It loads the data from files (such as Excel), organizes it into structured formats, and converts it into numerical tensors suitable for machine learning models. This module ensures that the data is clean, consistent, and properly formatted for training and analysis without requiring labeled information.

Model Training Module

This module is responsible for training the autoencoder model using baseline performance data. The model learns normal system behavior by minimizing reconstruction error between input and output data. It uses a lightweight neural network architecture implemented in PyTorch. Once training is complete, the model is saved for later use in drift detection, making the system efficient and reusable.

Drift Detection Module

The drift detection module is the core component of the system. It takes post-update performance data as input and passes it through the trained autoencoder model. The reconstruction error is calculated for each data point and compared against a predefined drift.

Reporting Module

This module generates detailed output after drift detection. It provides information such as the number of samples analyzed, number of detected drift points, and specific instances of anomalies. It presents results in a structured format, helping users understand whether the software update has introduced performance issues.

Visualization Module

The visualization module presents performance data in graphical form, such as bar charts comparing metrics before and after the update. This helps users quickly interpret changes in system behavior and identify trends or abnormalities. Visual representation improves usability and supports better decision-making.

User Interface Module

The user interface module provides an interactive dashboard for executing different stages of the system, such as data collection, model training, drift detection, and result visualization. It ensures ease of use by guiding users through the workflow in a step-by-step manner, making the system accessible even to non-technical users.

Model Management Module

This module is responsible for handling the storage, loading, and reuse of the trained autoencoder model. After the training phase, the model is saved in a file (e.g., .pth format) so that it can be reused for future drift detection without retraining. The module ensures that the correct model version is loaded during the detection phase and maintains consistency in analysis. It also enables flexibility for future enhancements such as updating or replacing the model when new data becomes available. This improves efficiency, reduces computational cost, and supports scalability of the system over time.

4.4 Methods and Algorithms

The DriftGuard system employs a highly optimized combination of statistical machine learning techniques to effectively detect drifts by analyzing complex system and network logs.

Autoencoder Model

The DriftGuard system primarily utilizes an **autoencoder**, which is a type of neural network designed for unsupervised learning. The autoencoder consists of two main components: an encoder and a decoder. The encoder compresses the input data into a lower-dimensional representation, capturing the essential features of system behavior, while the decoder reconstructs the original data from this compressed form. In this project, the model is trained using baseline performance metrics such as CPU usage, memory consumption, execution time, and derived ratios. The goal of training is to minimize the reconstruction error, allowing the model to accurately learn normal system behavior. Since the model is exposed only to normal data during training, it becomes highly sensitive to deviations, making it effective for anomaly detection.

Reconstruction Error-Based Detection

The core detection mechanism in DriftGuard is based on **reconstruction error**, which measures the difference between the original input data and the output generated by the autoencoder. After the model is trained, post-update performance data is passed through the autoencoder, and the reconstruction error is calculated using Mean Squared Error (MSE). If the input data follows normal patterns, the model can reconstruct it accurately, resulting in low error. However, if the system behavior changes due to a software update, the model fails to reconstruct the data effectively, leading to higher error values. This difference in error values helps identify anomalies and performance drift in the system.

Threshold-Based Decision Algorithm

To determine whether a data point represents normal behavior or drift, the system uses a threshold-based decision algorithm. A threshold is derived from the reconstruction error of baseline data, typically using statistical measures such as the mean or average error.

Data Collection Algorithm

The system incorporates a data collection algorithm that gathers system performance metrics at regular intervals using tools like psutil. The algorithm captures CPU usage, memory consumption, execution time, and calculates derived features such as CPU-to-memory ratio. These values are stored in a structured format and used as input for both training and detection phases. The consistency in data collection before and after updates ensures reliable comparison and accurate drift detection.

Model Training Algorithm

The training of the autoencoder is performed using a gradient descent optimization algorithm. During training, the input data is passed through the network, and the reconstruction loss is calculated. The model parameters are then updated using backpropagation to minimize this loss. This process is repeated for multiple epochs until the model achieves optimal performance. The trained model is then saved and reused during the drift detection phase, improving efficiency and reducing computational overhead.

Drift Detection Workflow Algorithm

The overall drift detection process follows a structured workflow algorithm. It begins with collecting baseline data and training the autoencoder model. After a software update, new performance data is collected and passed through the trained model. The reconstruction error is calculated and compared with the threshold to identify anomalies. Finally, the system generates a report indicating whether performance drift has occurred. This workflow ensures a systematic and automated approach to detecting changes in system behaviour

4.5 Structural Design

The structural design of the DriftGuard system is based on a modular and layered architecture that ensures flexibility, scalability, and ease of maintenance. The system is organized into independent components, each responsible for a specific function such as data collection, model training, drift detection, and reporting. These components interact through well-defined data flows, allowing the system to operate efficiently while maintaining clear separation of concerns. This design approach enables easy modification or extension of individual modules without affecting the overall system.The system follows a pipeline-based structure, where data flows sequentially through different stages. The process begins with the performance monitoring layer, which collects real-time system metrics using lightweight tools. This data is then passed to the data handling layer, where it is formatted and prepared for processing. The structured data is subsequently fed into the machine learning layer, where the autoencoder model is trained using baseline data. This pipeline ensures that each stage performs a dedicated task, improving system clarity and reliability.At the core of the structural design lies the autoencoder-based learning module, which acts as the intelligence of the system. The model is designed with a simple yet effective architecture consisting of an input layer, a compressed hidden layer, and an output layer. This structure allows the model to learn compact representations of normal system behavior and reconstruct input data efficiently. The model is trained using baseline data and then reused during the detection phase, ensuring consistency in evaluation. The use of a lightweight model architecture ensures low computational overhead, making the system suitable for desktop environments.

The system also includes a drift detection layer, which processes post-update data using the trained model. This layer calculates reconstruction error and compares it with a predefined threshold to determine whether performance drift has occurred.

The decision-making logic is simple yet effective, allowing the system to operate without complex rules or labeled datasets. This layer forms the core analytical component of the system, enabling automated anomaly detection.The final layer of the structural design is the output and visualization module, which presents the results to the user. This module generates detailed reports and graphical representations such as bar charts to compare performance metrics before and after updates. The system also includes a user interface that integrates all modules into a cohesive workflow, allowing users to execute each step sequentially. This enhances usability and ensures that the system can be effectively used even by non-technical users.Overall, the structural design of DriftGuard emphasizes modularity, simplicity, and efficiency. By organizing the system into well-defined layers and components, it ensures smooth data flow, accurate analysis, and easy extensibility. This design not only supports the current functionality but also provides a strong foundation for future enhancements such as real-time monitoring, advanced models, and integration with larger systems.In addition to its modular structure, the DriftGuard system is designed with a strong emphasis on loose coupling and high cohesion among its components. Each module performs a specific function independently, while communication between modules is handled through clearly defined data interfaces. This ensures that changes in one module, such as updating the model architecture or modifying the data collection process, do not significantly impact other components. Such a design improves system maintainability and supports future enhancements, including the integration of new algorithms or additional performance metrics without requiring major restructuring.

Furthermore, the structural design supports scalability and extensibility by allowing new functionalities to be incorporated with minimal effort. For instance, additional monitoring parameters, advanced machine learning models, or real-time processing capabilities can be integrated into the existing pipeline

4.6 System Design

The system design of DriftGuard follows a modular, pipeline-based architecture that organizes the entire workflow into clearly defined stages, including data collection, model training, drift detection, and reporting. Each component is designed to operate independently while maintaining a smooth flow of data between modules. The system begins with the performance monitoring module, which collects system metrics such as CPU usage, memory consumption, and execution time. This data is then processed and passed to the machine learning module, where the autoencoder model is trained to learn normal system behavior. The modular design ensures flexibility, allowing individual components to be modified or enhanced without affecting the overall system.

4.6.1 Input Design

The input design of the DriftGuard system is structured to ensure accurate, consistent, and reliable data collection for both training and drift detection phases. The system primarily accepts performance metrics collected from the host machine using monitoring tools such as psutil. These inputs include CPU usage, memory consumption, execution time, and derived features like CPU–memory ratio. The data is gathered at fixed intervals and stored in a structured format such as Excel files, ensuring uniformity across baseline (before update) and post-update datasets. This consistency is essential for meaningful comparison and effective anomaly detection.In addition to automated data collection, the input design ensures that the data is properly formatted and validated before being used by the model. The collected metrics are loaded using data processing libraries and converted into numerical tensors suitable for machine learning operations. Since the system follows an unsupervised approach, no labeled input is required, simplifying the data preparation process. The design also ensures that input data from different stages of the workflow remains compatible with the model, enabling smooth execution of training and drift detection without manual intervention.

4.6.1 Output Design

The output design of DriftGuard focuses on presenting results in a clear, structured, and user-friendly manner. The system generates detailed textual reports that summarize the analysis, including the number of samples processed, the number of detected drift points, and specific instances where anomalies occur. It also provides reconstruction error values and threshold comparisons, helping users understand the severity and nature of detected drift. This structured output enables developers and testers to quickly assess whether a software update has introduced performance issues.In addition to textual output, the system incorporates **visualization components** to enhance interpretability. Graphical representations such as bar charts are used to compare performance metrics before and after updates, making it easier to identify trends and deviations. The results are displayed through a user interface dashboard that guides users through each step of the process, from data collection to drift detection and visualization. This combination of analytical output and visual insights ensures that the system is not only accurate but also intuitive and accessible for effective decision-making.

Additionally, the output design of DriftGuard ensures clarity and usability by presenting results in a structured and easily interpretable format. Along with numerical summaries and visual charts, the system provides clear status messages indicating whether the software update is stable or has introduced performance drift. Each detected anomaly is highlighted with its corresponding reconstruction error, allowing users to pinpoint specific instances of concern. The output is designed to support quick decision-making by combining detailed analytical information with intuitive presentation, making it useful for both technical users and non-technical stakeholders.

4.7 Data Flow in the System

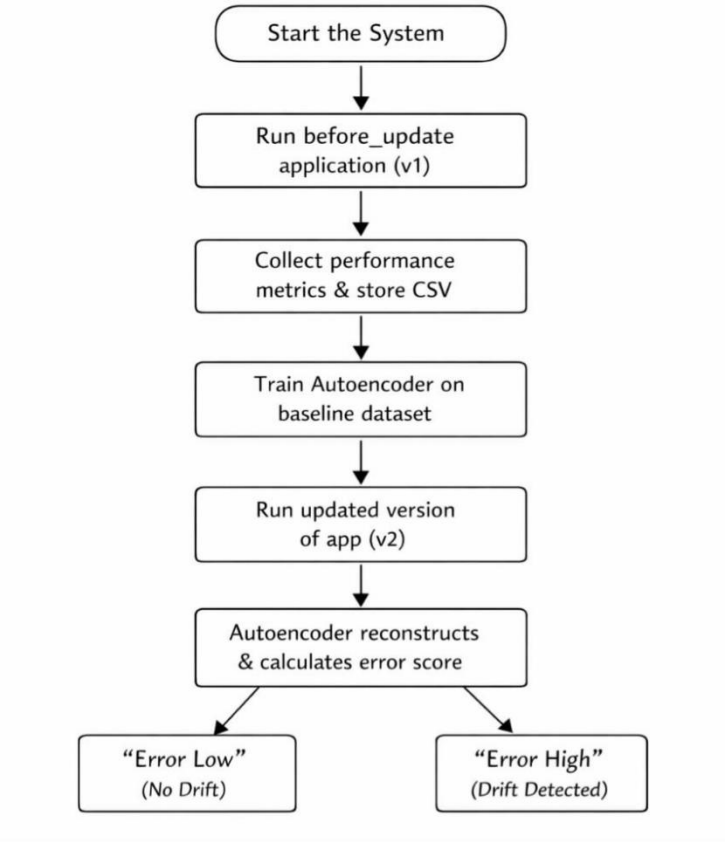


Fig 4.3 Data Flow Diagram

The Data Flow Diagram (DFD) of the DriftGuard system represents how data moves through different components of the system, from input collection to final output generation. It provides a clear visualization of how performance metrics are captured, processed, analyzed, and converted into meaningful insights. The DFD highlights the interaction between external entities (such as the user or system environment), processes (like data collection and drift detection), and data stores (such as baseline and post-update datasets). This helps in understanding the internal working of the system without focusing on implementation details.

At the initial stage, the DFD shows the input flow, where system performance data is collected using monitoring tools. This data includes CPU usage, memory consumption, execution time, and other relevant metrics. The collected data is then stored in structured formats, forming the baseline dataset before the update and the post-update dataset after changes are applied.

The next stage in the DFD represents the processing flow, where the baseline data is used to train the autoencoder model. Once trained, the model is stored and later used to process post-update data. The drift detection process takes this new data, reconstructs it using the trained model, and calculates reconstruction error. This error is compared against a predefined threshold to determine whether the system behavior has deviated from normal patterns.Finally, the DFD illustrates the output flow, where the results of drift detection are generated and presented to the user. The system produces reports indicating the presence or absence of drift, along with detailed information about detected anomalies. Visualization components further enhance this output by displaying performance comparisons through charts and graphs. Overall, the DFD provides a comprehensive view of how data flows through the DriftGuard system, ensuring transparency and clarity in its operation.The Data Flow Diagram can be further understood by considering its different levels of abstraction, typically represented as Level 0, Level 1, and Level 2 diagrams. At Level 0 (also called the context diagram), the entire DriftGuard system is shown as a single process interacting with external entities such as the user or system environment. This high-level view focuses on how input data enters the system and how results are delivered. At Level 1, the system is broken down into major processes such as data collection, model training, drift detection, and reporting. Level 2 provides a more detailed view of each process, showing internal operations like reconstruction, error calculation, and threshold comparison. This layered representation helps in understanding the system from both a broad and detailed perspective.Another important aspect of the DFD is the representation of data stores and their role in the system. DriftGuard uses separate data stores for baseline data, post-update data, and the trained model. These data stores ensure that information is preserved and can be reused at different stages of the workflow. For example, the baseline dataset is used during the training phase, while the trained model is reused during drift detection without requiring retraining. This separation of data not only improves efficiency but also enhances clarity by distinguishing between different types of information handled by the system.

4.8 UML Diagrams

4.8.1 Usecase Diagram

A Use Case Diagram represents the interaction between users and the DriftGuard system by illustrating how different actors, such as system administrators or engineers, interact with the system's functionalities. It highlights key use cases like monitoring system metrics, detecting anomalies, identifying drift, and viewing alerts on the dashboard. The diagram helps in understanding system requirements from a user perspective by clearly defining the roles, responsibilities, and interactions between the user and the system.

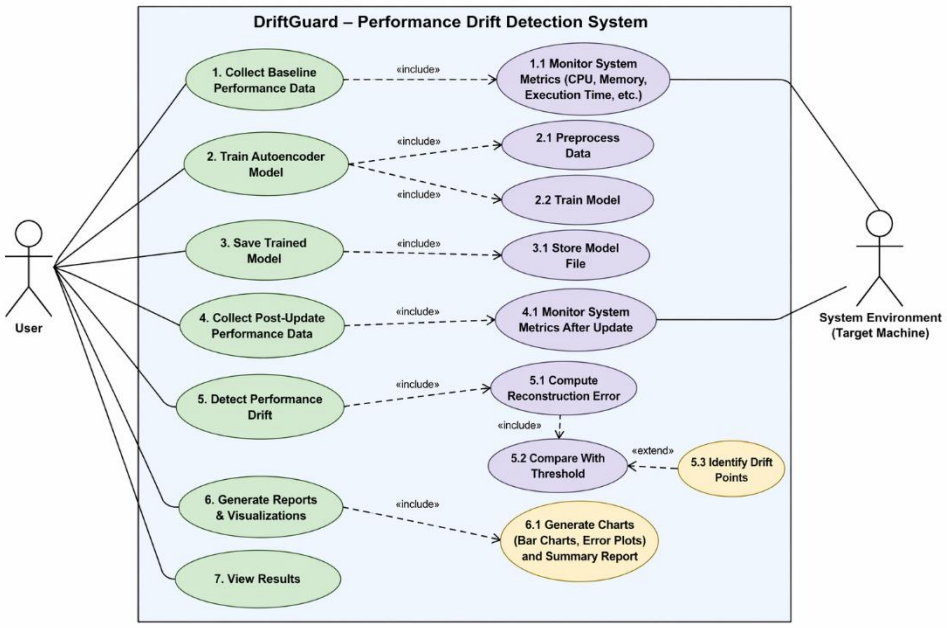


Fig 4.4 Usecase Diagram

4.8.2 Class Diagram

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is-a" or "has-a" relationship. Each class in the class diagram may be capable of providing certain functionalities. These functionalities provided by the class are termed "methods" of the class. Apart from this, each class may have certain "attributes" that uniquely identify the class.

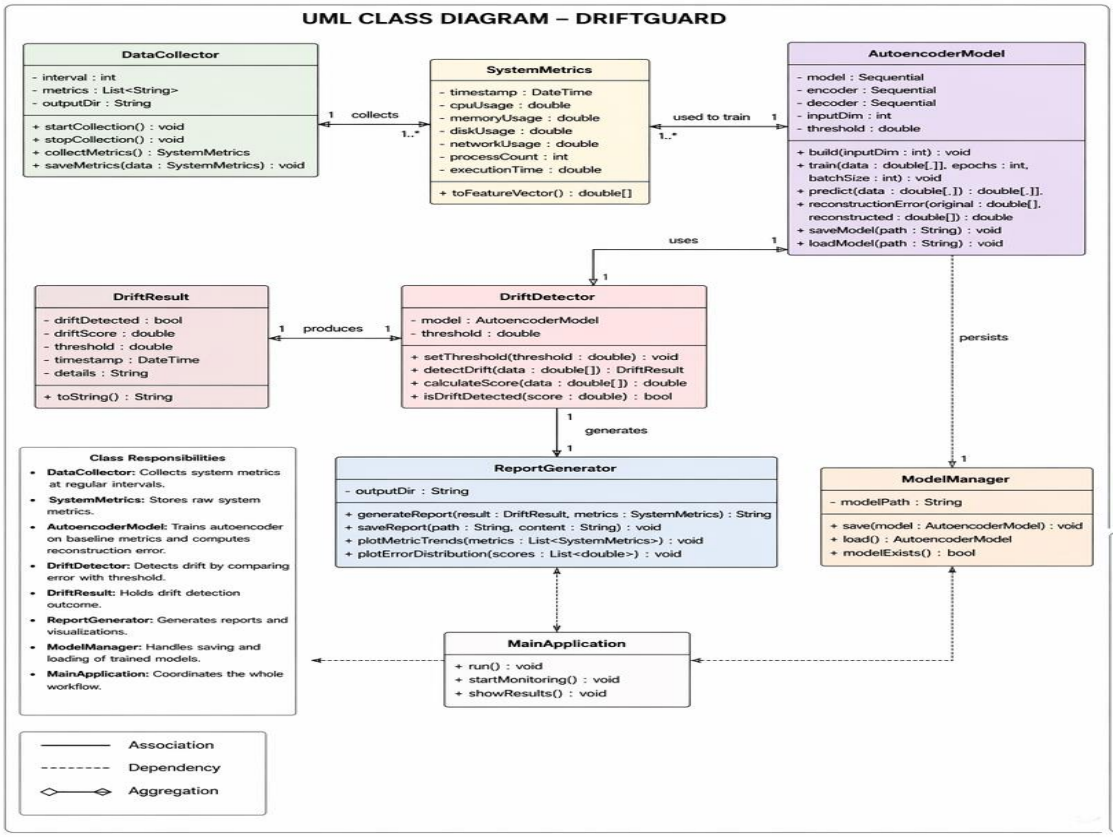


Fig 4.5 Class Diagram

the class diagram provides a structured representation of the system's internal components by organizing them into well-defined classes such as DataCollector, AutoencoderModel, DriftDetector, ReportGenerator, and UserInterface. Each class encapsulates specific attributes and methods relevant to its functionality—for example, the DataCollector class includes attributes like CPU, memory, and execution time, along with methods for collecting and storing metrics. Similarly, the AutoencoderModel class contains methods for training, saving, loading, and reconstructing data. The relationships between these classes, such as associations and dependencies, illustrate how data flows and operations are coordinated within the system. This detailed representation helps in understanding the system design, improving code organization, and ensuring efficient implementation of the DriftGuard architecture.

4.8.3 Sequence Diagram

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

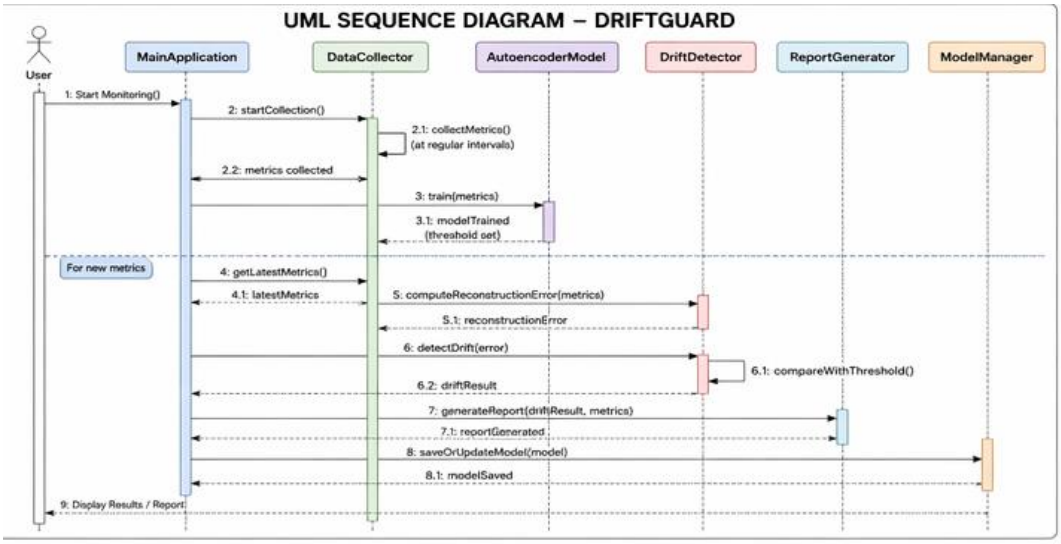


Fig 4.6 Sequence Diagram.

4.8.4 Activity Diagram

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration, and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by- step workflows of components in a system. An activity diagram shows the overall flow of control.

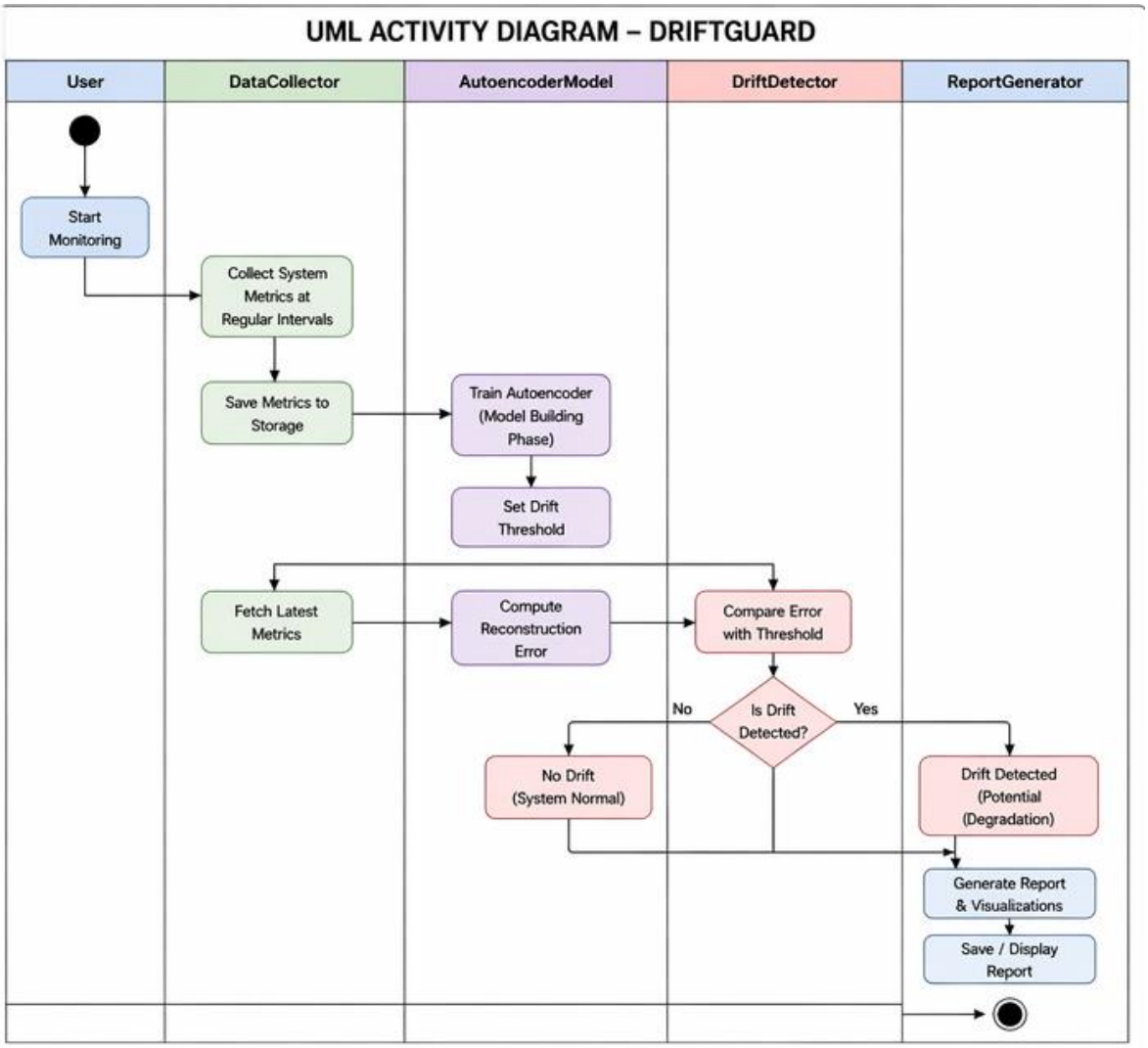


Fig 4.7 Activity Diagram

The activity diagram of the DriftGuard system illustrates the step-by-step workflow of the performance drift detection process, showing how different activities are executed in a sequential and logical manner. The process begins with the user initiating the system, followed by the collection of baseline performance data from the desktop application. Once the data is collected, the system proceeds to train the autoencoder model using this baseline data. After training, the system collects post-update performance data and feeds it into the trained model for analysis. The model reconstructs the input data, and the reconstruction error is calculated for each data point. A decision point is then used to compare the error with a predefined threshold to determine whether drift has occurred. If the error exceeds the threshold, the system identifies it as performance drift; otherwise, it is considered normal behavior.

4.8.5 Deployment Diagram

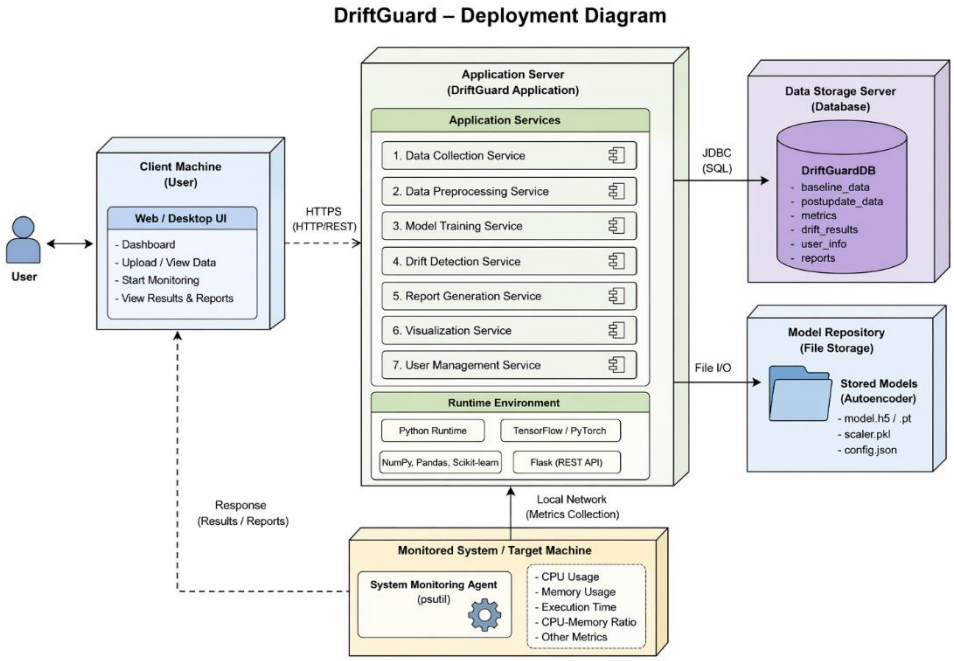


Fig 4.8 Deployment Diagram

The deployment diagram of the DriftGuard system illustrates how the application is physically distributed across different nodes and how its components interact in a runtime environment. It shows the client machine, where the user interacts with the system through a web or desktop interface, and the application layer, which hosts the core functionalities such as data collection, preprocessing, model training, drift detection, and report generation. The diagram also represents the monitored system (target machine), from which performance metrics are collected using a monitoring agent like psutil. These metrics are processed locally by the application without requiring external servers.

CHAPTER-5
TESTING

Need of Testing

The testing of the DriftGuard system was carried out using a manual testing approach, where each module was executed and validated step by step without the use of automated testing frameworks. Initially, the data collection process was manually triggered to verify that system performance metrics such as CPU usage, memory consumption, execution time, and derived ratios were being captured correctly. The generated datasets were inspected to ensure that the values were consistent, properly formatted, and stored accurately for further processing.In the next phase, the model training process was manually tested by running the training script with baseline data. The training logs, including epoch-wise loss values, were observed to confirm that the autoencoder model was learning effectively and that the loss was decreasing over time. The successful saving of the trained model file was also verified, ensuring that the model could be reused for drift detection without retraining.The drift detection functionality was then tested by manually providing post-update data to the system. Different scenarios were created by slightly modifying system performance conditions to simulate both normal and abnormal behavior. The output was carefully analyzed to check whether the reconstruction error was calculated correctly and whether the system accurately identified drift based on the predefined threshold. Special attention was given to verifying that both normal cases and anomaly cases were handled correctly.

Finally, the output and visualization components were manually validated by reviewing the generated reports and charts. The system's outputs, including the number of drift points, threshold values, and performance comparisons, were checked for correctness and clarity. The user interface was also tested to ensure smooth navigation through different steps such as data collection, model training, and result visualization. This manual testing approach helped confirm that the system functions correctly as a whole, even without the use of automated testing tools.

CHAPTER-6
CODE

Monitor.py

```
import psutil
import pandas as pd
import time
import os
def get_top_processes(limit=5):
 processes = []
 for p in psutil.process_iter(['name', 'cpu_percent', 'memory_percent']):
 try:
 processes.append({
 "name": p.info['name'],
 "cpu": p.info['cpu_percent'],
 "memory": p.info['memory_percent']
 })
 except (psutil.NoSuchProcess, psutil.AccessDenied):
 pass
 processes = sorted(processes, key=lambda x: x["cpu"], reverse=True)
 return processes[:limit]
def collect_data(filename):
 os.makedirs(os.path.dirname(filename), exist_ok=True)
 os.makedirs("reports", exist_ok=True)
 logs = []
 records = []
 logs.append(" Monitoring started")
 for i in range(25):
 start = time.time()
 cpu = psutil.cpu_percent(interval=0.3)
 memory = psutil.virtual_memory().percent
 exec_time = time.time() - start
 ratio = cpu / memory if memory else 0
```

```
records.append([cpu, memory, exec_time, ratio])
 logs.append(
 f"Sample {i+1}/25 → CPU:{cpu}% | Memory:{memory}% | Exec:{exec_time:.4f}"
 )
 df = pd.DataFrame(
 records,
 columns=["CPU", "Memory", "Exec_Time", "CPU_Memory_Ratio"]
 )
 df.to_excel(filename, index=False)
 logs.append(f" Data saved to {filename}")
 # ---------- SAVE TOP APPLICATIONS ----------
 top_apps = get_top_processes()
 logs.append(" Top applications affecting system performance:")
 if top_apps:
 pd.DataFrame(top_apps).to_excel(
 "reports/top_applications.xlsx", index=False
 )
 for app in top_apps:
 logs.append(
 f"{app['name']} → CPU {app['cpu']}% | Memory {round(app['memory'],2)}%"
 )
 else:
 logs.append("No high resource-consuming applications detected")
 return logs
```

Train_autoencoder.py

```
import pandas as pd
import torch
from torch import nn
from torch.optim import Adam
import os
class Autoencoder(nn.Module):
 def __init__(self):
 super().__init__()
 self.encoder = nn.Sequential(
 nn.Linear(4, 2),
 nn.ReLU()
 )
 self.decoder = nn.Sequential(
 nn.Linear(2, 4),
 nn.ReLU()
 )
 def forward(self, x):
 return self.decoder(self.encoder(x))
def train_autoencoder(data_path="data/before_update.xlsx"):
 df = pd.read_excel(data_path)
 data = torch.tensor(df.values, dtype=torch.float32)
 model = Autoencoder()
 criterion = nn.MSELoss()
 optimizer = Adam(model.parameters(), lr=0.001)
 logs = []
 for epoch in range(200):
 output = model(data)
 loss = criterion(output, data)
 optimizer.zero_grad()
 loss.backward()
 optimizer.step()
if epoch % 20 == 0:
```

```
 logs.append(f"Epoch {epoch}: Loss {loss.item():.6f}")
 os.makedirs("model", exist_ok=True)
 os.makedirs("reports", exist_ok=True)
 torch.save(model.state_dict(), "model/autoencoder.pth")
 # ---------- SAVE TRAINING LOGS ----------
 pd.DataFrame({"Training_Log": logs}).to_excel(
 "reports/training_logs.xlsx", index=False
 )
 logs.append(" Model trained & saved successfully")
 return logs
```

detect_drift.py

```
import pandas as pd
import torch
import os
from scripts.train_autoencoder import Autoencoder
def detect_drift():
 before = pd.read_excel("data/before_update.xlsx")
 after = pd.read_excel("data/after_update.xlsx")
 model = Autoencoder()
 model.load_state_dict(torch.load("model/autoencoder.pth"))
 model.eval()
 after_t = torch.tensor(after.values, dtype=torch.float32)
 with torch.no_grad():
 recon = model(after_t)
 errors = torch.mean((recon - after_t) ** 2, dim=1)
 threshold = errors.mean() + 2 * errors.std()

drift_samples = [
 (i + 1, round(err.item(), 6))
 for i, err in enumerate(errors)
 if err > threshold
 ]
```

```
# ---------- SAVE DRIFT RESULTS ----------
os.makedirs("reports", exist_ok=True)
pd.DataFrame(
drift_samples,
columns=["Sample", "Reconstruction_Error"]
).to_excel(
"reports/drift_results.xlsx", index=False
)
return {
"total": len(after),
"drifted": len(drift_samples),
"details": drift_samples
}
```

app.py

```
import streamlit as st
import os
import pandas as pd
from scripts.monitor import collect_data, get_top_processes
from scripts.train_autoencoder import train_autoencoder
from scripts.detect_drift import detect_drift
st.set_page_config(page_title="Sentinel – Drift Detection")
st.title(" Sentinel: Performance Drift Detection System")
def load_summary():
before = pd.read_excel("data/before_update.xlsx")
after = pd.read_excel("data/after_update.xlsx")

summary = pd.DataFrame({
 "Metric": ["CPU", "Memory", "Exec_Time"],
 "Before Update": [
 before["CPU"].mean(),
 before["Memory"].mean(),
 before["Exec_Time"].mean()
```

```
 ],
 "After Update": [
 after["CPU"].mean(),
 after["Memory"].mean(),
 after["Exec_Time"].mean()
 ]
 })
 os.makedirs("reports", exist_ok=True)
 summary.to_excel("reports/performance_summary.xlsx", index=False)
 return summary
# ---------- BEFORE UPDATE ----------
if st.button("Collect BEFORE Update Data"):
 with st.spinner("Collecting baseline metrics..."):
 logs = collect_data("data/before_update.xlsx")
 st.success("Baseline data collected")
 for log in logs:
 st.text(log)
st.divider()
# ---------- TRAIN MODEL ----------
if st.button("Train Model"):
 if not os.path.exists("data/before_update.xlsx"):
 st.error("Collect BEFORE update data first")
 else:
 with st.spinner("Training autoencoder..."):
 logs = train_autoencoder()
 st.success("Model trained")
 for log in logs:
 st.text(log)
 st.divider()
# ---------- AFTER UPDATE ----------
if st.button("Collect AFTER Update Data"):
 with st.spinner("Collecting updated metrics..."):
 logs = collect_data("data/after_update.xlsx")
```

```
 st.success("Updated data collected")
 for log in logs:
 st.text(log)
st.divider()
# ---------- DETECT DRIFT ----------
if st.button("Detect Drift"):
 result = detect_drift()
 st.subheader(" Drift Detection Result")
 st.metric("Total Samples", result["total"])
 st.metric("Drifted Samples", result["drifted"])
 if result["drifted"] == 0:
 st.success(" No performance drift detected")
 else:
 st.error(" Performance drift detected")
 for s, e in result["details"]:
 st.write(f"Sample {s} → Reconstruction Error: {e}")
st.divider()
# ---------- BAR CHART ----------
if st.button("Show Performance Bar Charts"):
 summary = load_summary()
 st.subheader(" Performance Comparison (Before vs After)")
 st.bar_chart(summary.set_index("Metric"))
 st.markdown("Detailed summary saved to `reports/performance_summary.xlsx”)
```

app_v1.py

```
import time
def run_app():
 for _ in range(25):
 lst = [i*i for i in range(50000)] # normal computation
 time.sleep(0.3) # normal delay
if __name__ == "__main__":
 print("Running BEFORE update app...")
 run_app()
```

app_v2.py

```
import time
def run_app():
 data = []
 for _ in range(25):
 data.append([i*i for i in range(60000)]) # slight load increase
 time.sleep(0.4) # hidden performance delay
if __name__ == "__main__":
 print("Running AFTER update (bug introduced)...")
 run_app()
```

```
            json_match = re.search(r'\{[\s\S]*\}', content)
```

6.1 System Implementation and Outputs

Home Page

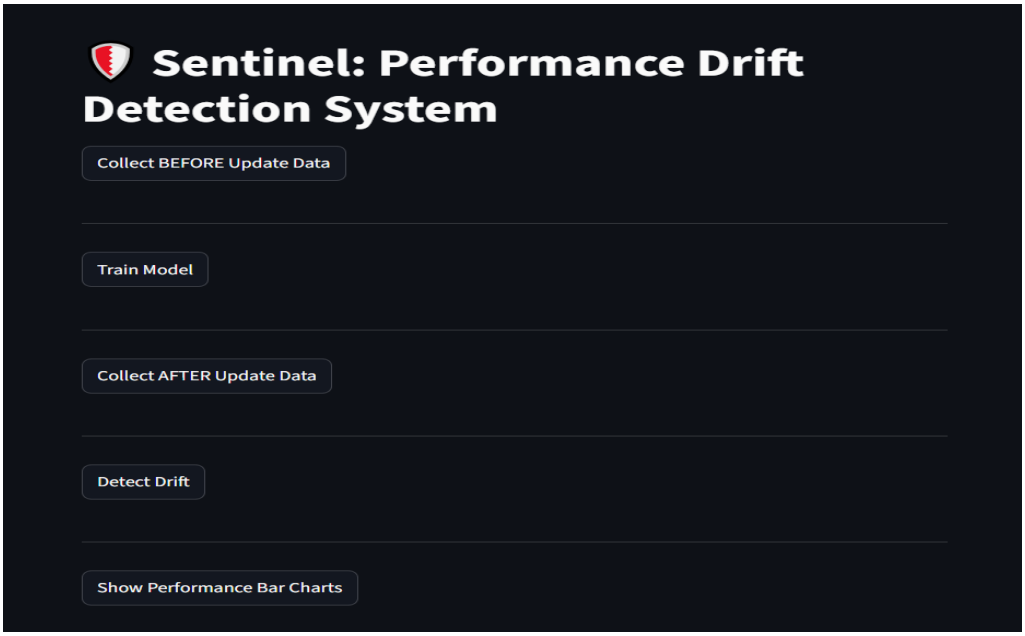


Fig 6.1.1 Home Page

The image represents the user interface of the DriftGuard (Sentinel) Performance Drift Detection System, designed as a streamlined dashboard to guide users through thecomplete drift detection workflow. The interface follows a sequential process, beginning with the collection of baseline performance data before a software update, followed by training an autoencoder model to learn normal system behavior. It then allows users to collect post-update performance data, which is analyzed to detect any deviations or drift using reconstruction error. Finally, the system provides visualization of results through performance bar charts, enabling users to easily interpret changes in system metrics. The clean, step-by-step layout ensures ease of use and clearly reflects the modular architecture of the underlying system.

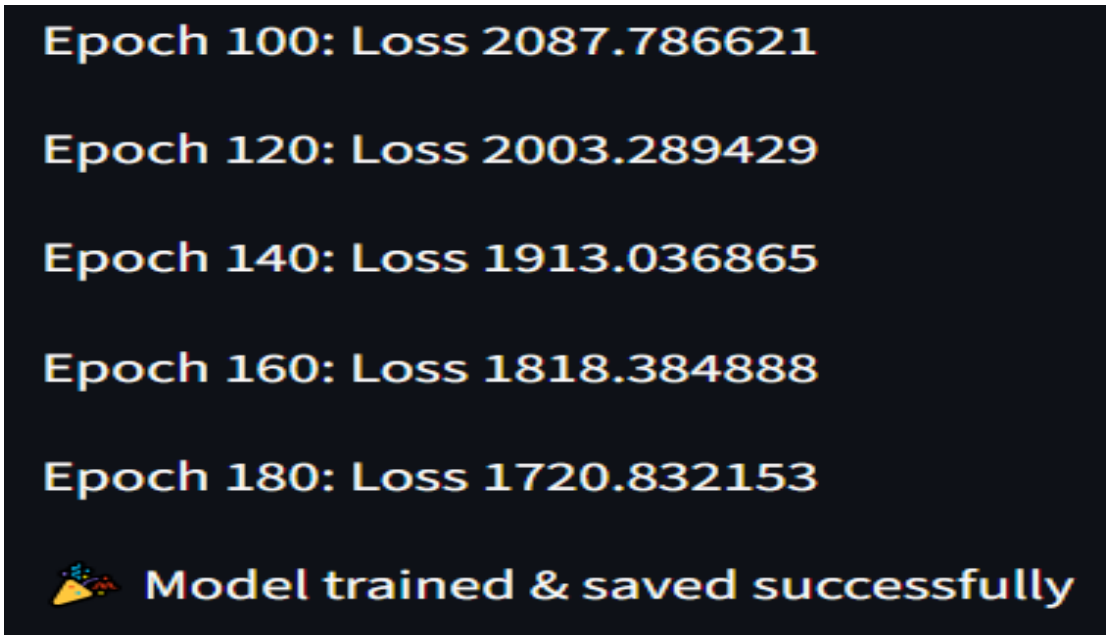


Fig 6.1.2 Autoencoder training

The image represents the training progress of the autoencoder model used in theDriftGuard system. It shows the loss values at different training epochs, indicatinghow the model improves over time. As the number of epochs increases (from 100 to180), the loss gradually decreases from 2087.78 to 1720.83, demonstrating that themodel is successfully learning the underlying patterns of the baseline performancedata. This reduction in loss reflects better reconstruction capability of theautoencoder. The final message confirms that the model training process has beencompleted and the trained model has been saved successfully for future driftdetection tasks.



Fig 6.1.3 Drift Detection

The image illustrates the output of the DriftGuard system after performing performance drift detection on post-update data. It presents a summary of the analysis, indicating that a total of 25 samples were evaluated, out of which 1sample was identified as exhibiting drift. This result suggests that while most of the system behavior remains consistent with the baseline, there is at least one instance of abnormal performance that deviates from the learned pattern. The system clearly highlights this condition with a warning mess age stating “Performance drift detected,” drawing attention to potential issues introduce dafter the software update. Additionally, the image provides detailed insight into the specific instance where drift was observed, showing that Sample 24 produced a reconstruction error of 1820.36, which exceeds the predefined threshold derived from baseline data. This indicates that the autoencoder model was unable to accurately reconstruct this sample, signaling anomalous behavior.Overall, this output demonstrates the effectiveness of the DriftGuard system inidentifying subtle performance deviations and providing both high-level summaries and detailed diagnostic information for further analysis.

Performance Comparison (Before vs After)

	Metric	color -- streamlit-generated	value -- streamlit-generated
0	CPU	Before Update	46.008
1	Memory	Before Update	85.928
2	Exec_Time	Before Update	0.316
3	CPU	After Update	31.084
4	Memory	After Update	87.224
5	Exec_Time	After Update	0.3147

Fig 6.1.4 Performance Comparison

The image presents a comparative analysis of system performance metrics before and after a software update, providing a clear view of how the update has impacted resource utilization. The table includes key metrics such as CPU usage, memory consumption, and execution time, with corresponding values recorded for both the “Before Update” and “After Update” scenarios. Prior to the update, the system shows a CPU usage of 46.008%, memory utilization of 85.928%, and execution time of 0.316 seconds. After the update, CPU usage decreases to 31.084%, indicating reduced processor load, while memory usage slightly increases to 87.224%,suggesting a marginal rise in memory consumption. The execution time remains almost consistent, with a slight decrease to 0.3147 seconds, implying that the update has not significantly affected processing speed. This comparison highlights that while the update may have optimized CPU usage, it has introduced minor changes in memory behavior. Such detailed metric-level insights are essential for understanding performance variations and complement the drift detection results by providing a quantitative basis for identifying and analyzing system changes.

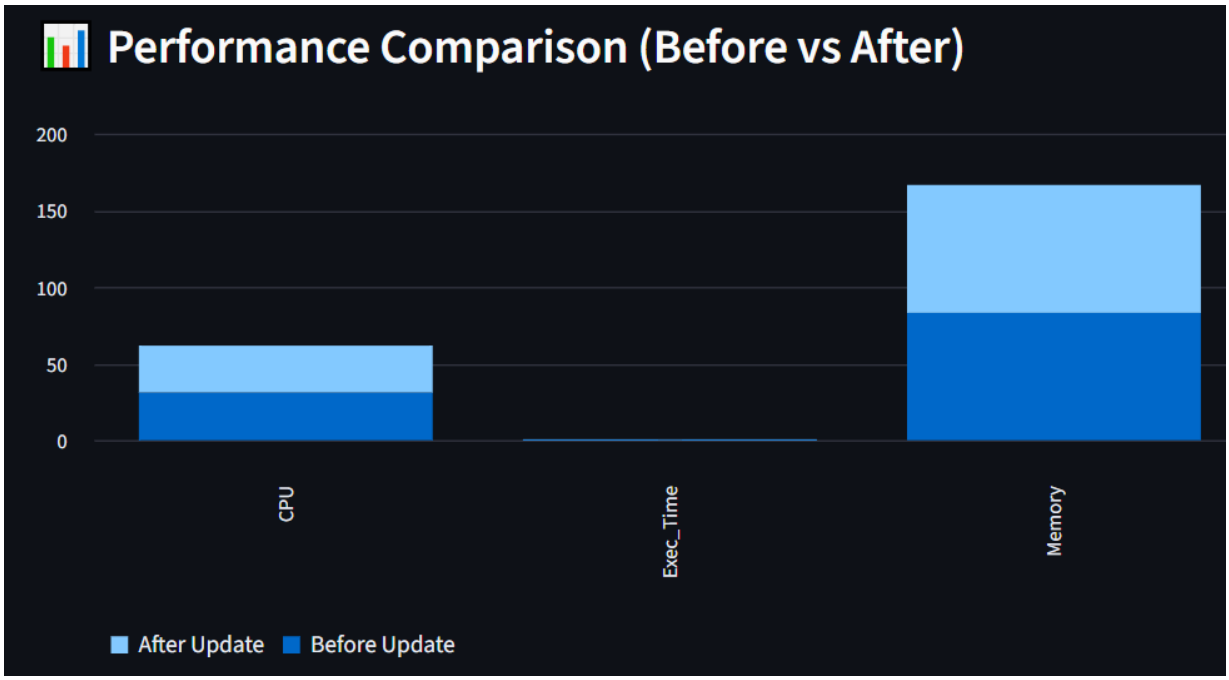


Fig 6.1.5 Comparison Graph

The image presents a visual comparison of system performance metrics before and after the software update using a bar chart representation. It illustrates three key parameters—CPU usage, execution time, and memory consumption—allowing for an intuitive understanding of how system behavior has changed post-update. Each metric is represented by two bars, distinguishing between “Before Update” and “AfterUpdate” values, making it easy to identify variations at a glance. From the chart, it can be observed that CPU usage has decreased after the update, indicating improved processing efficiency or reduced computational load. In contrast, memory usage has increased slightly, suggesting that the updated system may be utilizing more memory resources. The execution time remains relatively stable with only a negligible difference between the two states, implying that the update has not significantly impacted the speed of execution. Overall, this graphical representation complements the numerical comparison by providing a clear and concise visualization of performance changes, helping users quickly interpret system behavior and identify potential areas of concern or optimization following the update.

CHAPTER-7
RESULTS AND DISCUSSION

7.1 Results

The DriftGuard system was successfully implemented and evaluated using performance data collected before and after a software update. The autoencoder model was trained on baseline data representing normal system behavior, and its performance was observed through decreasing reconstruction loss during training. This indicated that the model effectively learned the underlying patterns of system metrics such as CPU usage, memory consumption, and execution time.During the drift detection phase, the system analyzed post-update data and computed reconstruction error for each sample. The results showed that most data points fell within the normal range, while a few instances exceeded the threshold, indicating potential performance drift. This demonstrates the system's ability to identify subtle deviations that may not be easily noticeable through manual observation.The generated reports provided a clear summary of the analysis, including the number of samples processed, the number of detected drift points, and the specific instances where anomalies occurred. The system also displayed detailed reconstruction error values, allowing users to understand the extent of deviation from normal behavior.

In addition to textual output, visualizations such as bar charts were used to compare performance metrics before and after the update. These graphs clearly highlighted changes in CPU usage, memory consumption, and execution time, making it easier to interpret system behavior. Overall, the results confirm that DriftGuard is effective in detecting performance drift in a simple and efficient manner.

7.2 Discussion

The results obtained from the DriftGuard system indicate that unsupervised learning using an autoencoder is a practical approach for detecting performance drift in desktop applications. The model was able to learn normal behavior without requiring labeled data, which simplifies implementation and makes the system suitable for real-world scenarios where labeled datasets are often unavailable.One important observation is that reconstruction error serves as a reliable indicator of system behavior. When the system operates under normal conditions, the autoencoder reconstructs input data with low error. However, when performance changes occur due to updates, the error increases significantly, allowing the system to detect anomalies effectively.However, the system's performance is influenced by the quality of baseline data. If the baseline does not represent all possible normal scenarios, the model may produce false positives or miss certain anomalies. Additionally, the use of a fixed threshold may not always adapt well to dynamic system conditions, suggesting the need for adaptive threshold mechanisms in future improvements.Despite these limitations, the system demonstrates strong potential as a lightweight and automated drift detection solution. Its simplicity, efficiency, and ability to operate without complex infrastructure make it suitable for standalone desktop environments. With further enhancements such as real-time monitoring and advanced models, the system can be extended to handle more complex scenarios.

7.3 Existing Systems vs Proposed System

Problems in Existing Systems

a. Dependence on static threshold-based monitoring

b. Inability to detect subtle or gradual performance drift

c. Requirement of labeled datasets or manual configuration

d. High complexity and reliance on cloud or large-scale infrastructure

e. Limited applicability to desktop or standalone environments

How DriftGuard Solves These Problems

a. Uses autoencoder-based unsupervised learning

b. Detects subtle anomalies using reconstruction error

c. Eliminates need for manual threshold tuning (data-driven threshold)

d. Designed as a lightweight, standalone system

e. Provides automated drift detection and visualization

CHAPTER-8
CONCLUSION

8.1 Project Conclusion

The DriftGuard system demonstrates an effective and practical approach to detecting performance drift in desktop applications after software updates. By leveraging an unsupervised autoencoder model, the system successfully learns baseline behavior from pre-update data and identifies anomalies using reconstruction error without requiring labeled datasets. The integration of real-time performance monitoring with lightweight model architecture ensures that the system remains efficient and suitable for resource-constrained environments. Experimental results show that DriftGuard can detect subtle deviations in system metrics such as CPU usage, memory consumption, and execution time, thereby helping identify hidden performance issues that may not be immediately visible through traditional testing methods.

In addition to accurate drift detection, DriftGuard provides clear visualization and reporting features that enhance interpretability and decision-making. The system's modular design allows easy integration into testing workflows and can be extended to support real-time monitoring or advanced anomaly detection techniques in the future. Although the current implementation focuses on performance metrics, the framework has the potential to be expanded for broader applications, including security anomaly detection and large-scale systems. Overall, DriftGuard offers a scalable, efficient, and user-friendly solution for ensuring software reliability and performance stability after updates.Its ability to work with unlabeled data and operate in a standalone desktop setup makes it highly adaptable across different domains and applications. With further enhancements such as real-time alerting, explainability features, and integration with CI/CD pipelines, DriftGuard can evolve into a comprehensive performance assurance tool, contributing significantly to the development of robust and reliable software systems.

Future Scope

The future scope of the DriftGuard system can be significantly expanded by enhancing its capability to handle real-time and continuous monitoring scenarios. Currently, the system operates in a batch-based manner by comparing pre-update and post-update data; however, integrating streaming data processing techniques would allow DriftGuard to detect performance drift as it occurs. This can be achieved by incorporating real-time data pipelines and stream-based drift detection algorithms, enabling continuous observation of system behavior. Such an extension would be highly beneficial in production environments where immediate detection of anomalies is critical to maintaining system reliability and user experience.

Another important direction for future improvement lies in advancing the underlying machine learning models used for drift detection. While the current implementation uses a basic autoencoder, more sophisticated models such as Variational Autoencoders (VAEs), LSTM-based autoencoders, or hybrid approaches combining multiple algorithms (e.g., Autoencoder + Isolation Fores can be explored to improve detection accuracy and robustness. Additionally, integrating explainability techniques such as SHAP or feature importance analysis would provide deeper insights into which performance metrics contribute most to the detected drift. This would make the system not only a detection tool but also a diagnostic aid for developers and system administrators.The addition of a more advanced user interface with dashboards, alerts, and reporting features along with support for cloud and distributed environments would enhance usability and scalability. Future work could also explore expanding the system's scope beyond performance monitoring to include security anomaly detection, making DriftGuard a versatile tool for ensuring both performance stability and system integrity

REFERENCES

[1] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” ACMComputing Surveys, vol. 41, no. 3, pp. 1–58, 2009.[2] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey onconcept drift adaptation,” ACM Computing Surveys, vol. 46, no. 4, pp. 1–37, 2014.

[3] F. T. Liu, K. M. Ting, and Z. H. Zhou, in Proceedings of the IEEE InternationalConference on Data Mining, 2008, pp. 413–422.

[4] R. Chalapathy and S. Chawla, “Deep learning for anomaly detection: A survey,”arXiv preprint arXiv:1901.03407, 2019.

[5] A. Bifet and R. Gavaldà, “Learning from time-changing data with adaptivewindowing,” in Proceedings of the SIAM International Conference on Data Mining,2007, pp. 443–448.

[6] S. M. Lundberg and S. I. Lee, “A unified approach to interpreting modelpredictions,” in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[7] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MITPress, 2016.(Foundational deep learning textbook — directly supports your Autoencoder modeldesign and training)

[8] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data withneural networks," Science, vol. 313, no. 5786, pp. 504–507, 2006.(Original autoencoder compression work — core theoretical support for yourAutoencoder-based anomaly detection)

[9] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, "Long Short Term Memory Networksfor anomaly detection in time series," in Proceedings of the European Symposium onArtificial Neural Networks (ESANN), 2015, pp. 89–94.(LSTM for streaming time series anomaly detection — relevant to your continuoustelemetry analysis)

[10] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis foranomaly detection," in Proceedings of the IEEE International Symposium onSoftware Reliability Engineering (ISSRE), 2016, pp. 207–218.(Log-based anomaly detection — directly supports your application log telemetry collection.

[11] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection anddiagnosis from system logs through deep learning," in Proceedings of the ACMSIGSAC Conference on Computer and Communications Security (CCS), 2017, pp.1285–1298. (Deep learning on system logs — closely aligned with your log stream anomalydetection pipeline)

[12] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scalesystem problems by mining console logs," in Proceedings of the ACM Symposium onOperating Systems Principles (SOSP), 2009, pp. 117–132.(Console log mining for production system issues — directly relevant to yourtelemetry and log processing module)

[13] I. Žliobaitė, "Learning under concept drift: An overview," arXiv preprintarXiv:1010.4784, 2010.(Comprehensive concept drift survey — strongly supports your Section III onadaptive learning and problem formalisation)

[14] M. Baena-Garcia, J. del Campo-Avila, R. Fidalgo, A. Bifet, R. Gavaldà, and R.Morales-Bueno, "Early drift detection method," in Proceedings of the ECML/PKDDWorkshop on Knowledge Discovery from Data Streams, 2006, pp. 77–86.(EDDM drift detection — relevant comparison method to your drift detectiontaxonomy in Section IV)

[15] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Macias-Fernandez, and E. Vazquez,"Anomaly-based network intrusion detection: Techniques, systems and challenges,"Computers & Security, vol. 28, no. 1–2, pp. 18–28, 2009.(Anomaly-based intrusion detection — directly supports your security anomalydetection scenarios)

[16] A. L. Buczak and E. Guven, "A survey of data mining and machine learningmethods for cyber security intrusion detection," IEEE Communications Surveys &Tutorials, vol. 18, no. 2, pp. 1153–1176, 2016.(ML methods for cybersecurity — supports your security-focused anomaly detectionmotivation and results)

[17] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson,"Estimating the support of a high-dimensional distribution," Neural Computation,vol. 13, no. 7, pp. 1443–1471, 2001.(One-Class SVM for novelty detection — supports your unsupervised anomalydetection design rationale)

[18] M. Goldstein and S. Uchida, "A comparative evaluation of unsupervised anomalydetection algorithms for multivariate data," PLOS ONE, vol. 11, no. 4, e0152173, 2016.(Comparative study of unsupervised detectors — directly supports your hybriddetector combination and design choices)

[19] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom,"Detecting spacecraft anomalies using LSTMs and nonparametric dynamicthresholding," in Proceedings of the ACM SIGKDD International Conference onKnowledge Discovery and Data Mining, 2018, pp. 387–395. (Dynamic thresholdinginstead of fixed rules — directly supports your argument against threshold-basedmonitoring)

[20] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD:UnSupervised Anomaly Detection on multivariate time series," in Proceedings of theACM SIGKDD International Conference on Knowledge Discovery and Data Mining,2020, pp. 3395–3404.(Unsupervised anomaly detection on multivariate streams —directly aligned with your multi-metric CPU, memory, execution time analysis)

[21] N. Laptev, S. Amizadeh, and I. Flint, "Generic and scalable framework forautomated time-series anomaly detection," in Proceedings of the ACM SIGKDDInternational Conference on Knowledge Discovery and Data Mining, 2015, pp. 1939–1947.Scalable real-time anomaly detection — supports your system architecturedesign for live deployments)

[22] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in Proceedings of the ACM SIGKDD InternationalConference on Knowledge Discovery and Data Mining, 2016, pp. 1135–1144.(LIME explainability method — supports your explainability section as a relatedinterpretability approach)

[23] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machinelearning," arXiv preprint arXiv:1702.08608, 2017.(Foundations of interpretable ML — supports the theoretical motivation behind yourexplainability module.